Vocabulary Types

The small standard types that make a function's contract visible in its signature: `pair` and `tuple`, `optional`, `variant`, `expected` and `span`. Builds on Types, Initialization & Casts.

Why vocabulary types

A signature like `int find(const char* s)` hides its contract: does `-1` mean "not found"? Is `s` allowed to be null? Who owns the result? A **vocabulary type** puts the answer in the type, so every reader and the compiler see it.

THE QUESTION	THE OLD ANSWER	THE TYPE
Two or three values back from one function?	out-parameters, a throwaway struct	`std::pair`, `std::tuple`, or a named struct
A value that may be absent?	a magic value (`-1`), a null pointer	`std::optional<T>`
One of several different types?	a `union` plus a tag, a base class	`std::variant<A, B>`
A value, or the reason there is none?	an error code plus an out-parameter, an exception	`std::expected<T, E>` (C++23)
A run of elements?	a pointer plus a count	`std::span<T>`, `std::string_view`

All of them are ordinary value types: no heap allocation, no virtual functions, nothing to free.

`pair`, `tuple` and structured bindings

`std::pair<A, B>` holds two values, `std::tuple<A, B, C ...>` any number. Structured bindings (C++17) unpack them into named variables.

```
std::pair<int, int> minmax(const std::vector<int>& v);      // returns {smallest, largest}
auto [lo, hi] = minmax({3, 9, 1, 7});                       // lo = 1, hi = 9: two names instead of .first / .second

auto t = std::make_tuple(1, 2.5, std::string("x"));
std::get<1>(t);                    // 2.5   (index must be a compile-time constant)
std::get<std::string>(t);          // "x"   (or by type, when the type appears once)

std::map<std::string, int> m{{"a", 1}};
auto [it, inserted] = m.insert({"a", 2});   // insert returns pair<iterator, bool>
// inserted == false, it->second == 1: the key existed, nothing changed
```

- A pair or tuple has **no names**: `.first` and `std::get<2>` say nothing about meaning. For a result that leaves a function and is used in more than one place, a small named `struct` reads better.
- Comparison is built in and lexicographic: pairs and tuples compare element by element, so they work as `map` keys and in `std::sort`.
- `std::tie(a, b) = f();` assigns into existing variables; `std::ignore` skips one.
- Structured bindings also unpack plain structs and arrays: `auto [x, y] = point;`. They are copies unless you write `auto&` or `const auto&`.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 17.1 The tuple type (PDF pages 747 to 751)

`optional`: a value or nothing

`std::optional<T>` holds either a `T` or nothing. The "nothing" is part of the type, so a caller cannot forget it exists.

```
std::optional<int> parseInt(std::string_view s) {
    int v = 0;
    auto r = std::from_chars(s.data(), s.data() + s.size(), v);
    if (r.ec ≠ std::errc{} || r.ptr ≠ s.data() + s.size()) return
std::nullopt;   // "no value"
    return v;
// converts to optional<int>
}

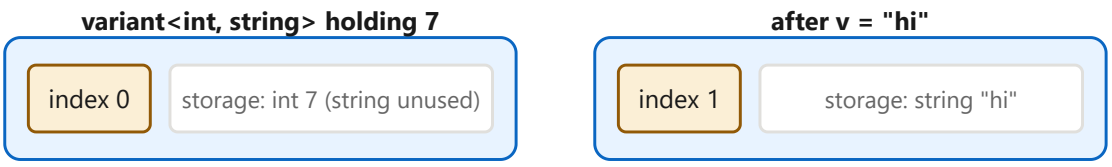
if (auto n = parseInt("42")) use(*n);        // test, then dereference: n is
true when a value is present
parseInt("4x").value_or(-1);                 // -1: a default for the empty
case
parseInt("").has_value();                    // false
std::optional<int> e;  e.value();            // throws
std::bad_optional_access
*e;                                          // undefined behaviour: no
check, like dereferencing a null pointer
```

OPERATION	ON AN EMPTY OPTIONAL
`if (o)` / `o.has_value()`	false
`*o`, `o→member`	undefined behaviour: check first
`o.value()`	throws `bad_optional_access`
`o.value_or(x)`	returns `x`
`o.reset()`, `o = std::nullopt`	makes it empty

- The value lives **inside** the optional (compiled: `sizeof(int)` is 4, `sizeof(optional<int>)` is 8: the value plus a flag, padded). No heap allocation, unlike a pointer or `unique_ptr`.
- Use it for a result that is *legitimately* absent: a lookup that may miss, a parse of untrusted input, an optional configuration value. It does not say *why* the value is missing; for that, use `expected`.
- Not a replacement for a reference or pointer that means "refers to an object owned elsewhere": `optional<T&>` does not exist. Use a pointer there.
- Marking a function `[[nodiscard]]` keeps callers from dropping the result.

`variant` and `std::visit`

`std::variant<A, B, C>` is a type-safe union: it holds exactly one of its alternatives and knows which.



Size: the largest alternative plus the index. No heap allocation for the variant itself.

One alternative at a time; the index says which.

```
std::variant<int, std::string> v = 7;
v.index();                                  // 0: the int is active
std::holds_alternative<int>(v);             // true
std::get<int>(v);                           // 7
std::get<std::string>(v);                   // throws std::bad_variant_access:
the wrong alternative
if (auto* s = std::get_if<std::string>(&v)) { }   // a pointer, or nullptr:
no exception
v = std::string("hi");                      // now index 1; the int is gone
std::variant<int, std::string> d;           // default: holds a value-initialised
FIRST alternative (index 0, int 0)
```

Visiting every alternative

`std::visit` calls a function with whichever alternative is active. Give it a callable that handles **every** alternative, and a missing one is a **compile error**, not a silent bug.

```
template <class... Ts> struct overloaded : Ts... { using Ts::operator()...; };   // glue lambdas into one callable
template <class... Ts> overloaded(Ts...) -> overloaded<Ts...>;
// deduction guide (needed before C++20)

struct Circle { double r; };
struct Rect   { double w, h; };
using Shape = std::variant<Circle, Rect>;           // no base class, no virtual functions

double area(const Shape& s) {
    return std::visit(overloaded{
        [](const Circle& c) { return 3.0 * c.r * c.r; },
        [](const Rect& r)   { return r.w * r.h; }
    }, s);
}
std::vector<Shape> shapes{Circle{2}, Rect{3, 4}};   // a heterogeneous collection stored by value
// total area: 12 + 12 = 24 (compiled)
```

- **Variant or virtual functions?** A variant fits a *closed* set of types you control (messages, tokens, syntax-tree nodes), where you keep adding operations. Virtual functions fit an *open* set that others may extend. The trade-off is spelled out under Visitor in More Patterns.
- If an alternative's constructor throws during assignment, the variant can end up *valueless by exception* (`valueless_by_exception()`). It is rare, and it is why you should keep alternatives cheap and non-throwing to move.
- A reference alternative is ill-formed (use `std::reference_wrapper`). The same type twice is legal, but then you can only reach it by index, not by type.
- `std::monostate` is an empty alternative: put it first to make a variant default-constructible when no alternative should be the default.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 19.5 to 19.6 Nested classes and unions (PDF pages 872 to 880)

`expected` : a value or an error

`std::expected<T, E>` (C++23) holds either the result `T` or an error `E`. It is `optional` that explains *why* the value is missing, and an alternative to exceptions for failures that are normal.

```
enum class ParseError { Empty, NotANumber };

std::expected<int, ParseError> parse(std::string_view s) {
    if (s.empty()) return std::unexpected(ParseError::Empty);        // the
error path
    if (auto n = parseInt(s)) return *n;                             // the
value path
    return std::unexpected(ParseError::NotANumber);
}

auto r = parse("12");
if (r) use(*r);                                  // true: it holds a value
else   report(r.error());                        // otherwise: the error
parse("zz").value_or(0);                         // 0: a default when it holds
an error
parse("").value();                               // throws
std::bad_expected_access<ParseError>
```

FAILURE STYLE	COST AND VISIBILITY	USE FOR
Exception	invisible in the signature; a throw is slow	rare failures the caller cannot handle here, constructors
Error code plus out-parameter	easy to ignore, clumsy to write	C APIs
`optional<T>`	visible, free, says nothing about why	"not found", where the reason is obvious

`expected<T, E>`	visible, free, carries the reason	expected failures: parsing, I/O, validation

It needs a C++23 standard library (GCC 12 or later, recent MSVC). With an older one, `std::variant<T, E>` or a small result struct does the same job, and the free `tl::expected` library has the same interface. Mark the function `[[nodiscard]]` so the result cannot be ignored.

`span` and `string_view`: non-owning views

A **view** is a pointer plus a length: it looks at elements that someone else owns. `std::span<T>` (C++20) views any contiguous run of `T`; `std::string_view` (C++17) is the same idea for characters.

```
long sum(std::span<const int> xs) { long t = 0; for (int x : xs) t += x;
return t; }

std::vector<int> vec{1, 2, 3, 4, 5};
int arr[3] = {10, 20, 30};
sum(vec);                              // 15: a vector converts to a span
with no copy
sum(arr);                              // 60: so does a C array, and its size
is remembered
sum(std::span(vec).subspan(1, 3));     // 9:  elements 2, 3, 4: a slice is
just a new pointer and length
sum(std::span(vec).first(2));          // 3
```

- A span is two words (compiled: 16 bytes on a 64-bit machine). Pass it **by value**.
- `span<const int>` is read-only; `span<int>` lets the function modify the elements in place. It still cannot grow or shrink the container.
- It replaces the `(T* data, size_t n)` pair. Overflows are easier to spot, and a function written against `span` accepts a vector, an array, a `std::array` or a part of any of them.
- **Dangling.** A view must not outlive what it looks at. A `span` of a vector that then reallocates (`push_back`), or a `string_view` of a temporary string, points at freed memory. Use views for parameters and short-lived local use, not for members that live longer than the data.

> **RULE OF THUMB** Take a view to *read or modify* existing elements; take or return the owning type (`vector`, `string`) to *keep* them.

Which one when

SITUATION	USE
Return two related values that are used right away	`pair` / structured bindings; a named `struct` if they travel further
A lookup that may find nothing	`optional<T>`
A parse or I/O call whose failure has a reason	`expected<T, E>` (or an exception if it is rare)
A value that is one of a few known types	`variant` plus `visit`
A function that reads a run of elements	`span<const T>`; `string_view` for text
A value of any type, unknown at compile time	`std::any` (rare: you lose compile-time checking; prefer a variant)

The same idea runs through all of them: make illegal or missing states **explicit in the type** so the compiler, not a comment, enforces them.

Cheat sheet

Answer aloud, then open.

<details>
<summary>▼ What is a vocabulary type?</summary>

A small standard type that expresses a common idea in a signature: `optional` (maybe a value), `variant` (one of several types), `expected` (a value or an error), `span` and `string_view` (a non-owning view). They replace magic values, tagged unions and pointer-plus-count pairs, so the contract is visible and checked.
</details>

▼ `optional` versus a pointer for "may be absent"?

`optional<T>` holds the value itself (no heap allocation, no ownership question) and is empty when absent. A pointer means "refers to something owned elsewhere" and can be null. Use `optional` for a possibly missing *value*, a pointer for a possibly missing *reference*.

▼ What happens on `*opt`, `opt.value()` and `opt.value_or(x)` when it is empty?

`*opt` is undefined behaviour, `value()` throws `std::bad_optional_access`, and `value_or(x)` returns `x`. Test with `if (opt)` first when you can.

▼ How does `std::variant` differ from a `union`?

A variant remembers which alternative is active (`index()`), destroys the old one and constructs the new one on assignment, works with non-trivial types such as `std::string`, and throws `bad_variant_access` if you read the wrong one. A raw union does none of that, and reading the wrong member is undefined behaviour.

▼ Why is `std::visit` safer than a chain of `if` / `get_if`?

The visitor must handle every alternative, so adding a type to the variant makes every incomplete visitor a compile error instead of a silent missing case. It also reads the active alternative without a manual check.

▼ Variant or virtual functions for a set of message or shape types?

A variant when the set of types is closed and you keep adding operations: values are stored inline, there are no allocations or virtual calls, and the compiler checks completeness. Virtual functions when the set of types is open for others to extend and the operations are few.

▼ `optional`, `expected`, exception or error code?

`optional` when "no value" needs no explanation. `expected<T, E>` (C++23) when the failure is normal and the reason matters. An exception for rare failures the local code cannot handle, and for constructors. An error code only at C boundaries.

▼ What does `std::span` replace, and what is its danger?

It replaces a `(pointer, count)` pair: a non-owning view of contiguous elements that remembers its size and accepts a vector, an array or a part of either, without copying. Like `string_view` it dangles if the data is destroyed or reallocated while the view is alive.

▼ How do you return several values from a function?

A `std::pair` or `std::tuple`, unpacked with structured bindings (`auto [a, b] = f();`), or a small named struct when the fields need names or the result travels. Not out-parameters, except where you want to reuse a buffer.